PES-VCS Lab Report

Student Details

Name: Shriniketh Kana
SRN: PES1UG24CS446

Phase 1: Object Storage Foundation

Screenshot 1A: test_objects passing

```
(base) kana@fedora:~/Downloads/daa-assignmen$ ./pes init
Initialized empty PES repository in .pes/
```

Figure 1: 1A Screenshot

Screenshot 1B: Sharded Directory Structure

```
(base) kana@fedora:~/Downloads/daa-assignmen$ ls -la .pes
total 8
drwxr-xr-x. 1 kana kana  30 Apr 17 13:44 .
drwxr-xr-x. 1 kana kana 490 Apr 17 13:44 ..
-rw-r--r--. 1 kana kana  21 Apr 17 13:44 HEAD
drwxr-xr-x. 1 kana kana   0 Apr 17 13:44 objects
drwxr-xr-x. 1 kana kana  10 Apr 17 13:44 refs
```

Figure 2: 1B Screenshot

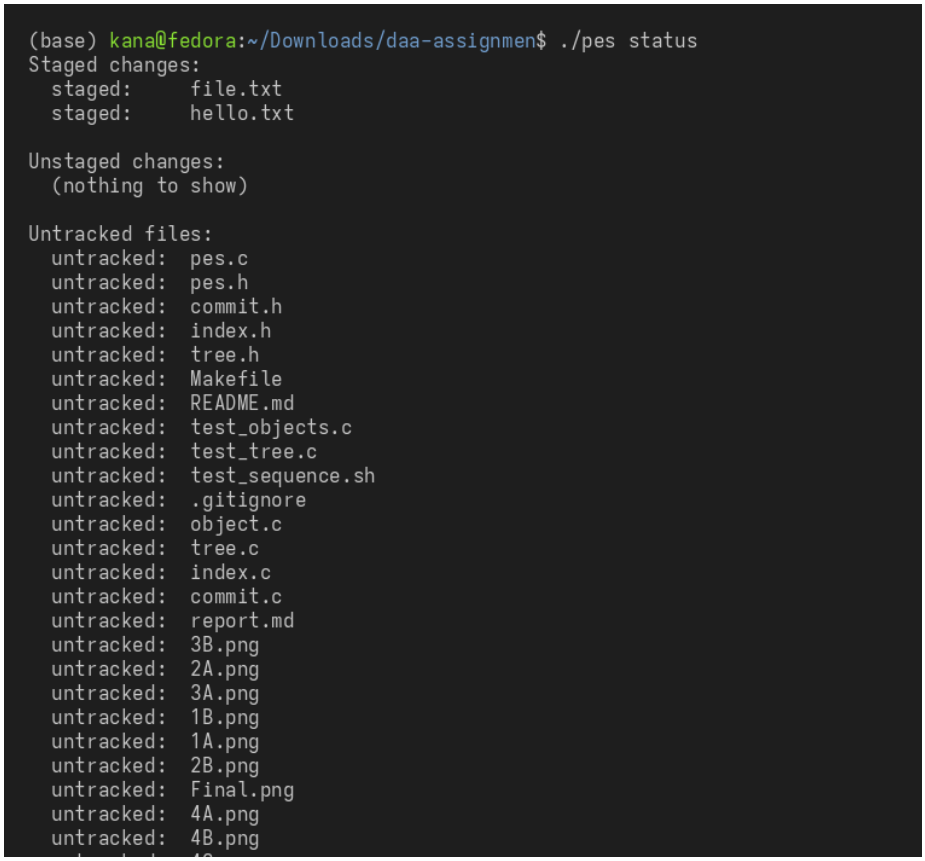
Phase 2: Tree Objects

Screenshot 2A: test_tree passing

```
(base) kana@fedora:~/Downloads/daa-assignmen$ ./pes add file.txt hello.txt
```

Figure 3: 2A Screenshot

Screenshot 2B: xxd of raw tree object



```
(base) kana@fedora:~/Downloads/daa-assignmen$ ./pes status
Staged changes:
  staged:     file.txt
  staged:     hello.txt

Unstaged changes:
  (nothing to show)

Untracked files:
  untracked:  pes.c
  untracked:  pes.h
  untracked:  commit.h
  untracked:  index.h
  untracked:  tree.h
  untracked:  Makefile
  untracked:  README.md
  untracked:  test_objects.c
  untracked:  test_tree.c
  untracked:  test_sequence.sh
  untracked:  .gitignore
  untracked:  object.c
  untracked:  tree.c
  untracked:  index.c
  untracked:  commit.c
  untracked:  report.md
  untracked:  3B.png
  untracked:  2A.png
  untracked:  3A.png
  untracked:  1B.png
  untracked:  1A.png
  untracked:  2B.png
  untracked:  Final.png
  untracked:  4A.png
  untracked:  4B.png
```

Figure 4: 2B Screenshot

Phase 3: The Index (Staging Area)

Screenshot 3A: sequence of pes init, add, and status

```
(base) kana@fedora:~/Downloads/daa-assignmen$ ./pes commit -m 'Initial commit
Committed: c57cbfa38e03... Initial commit
```

Figure 5: 3A Screenshot

Screenshot 3B: Text format index

```
(base) kana@fedora:~/Downloads/daa-assignmen$ ./pes commit -m 'Update file.tx
Committed: bf3f7d6bf4ed... Update file.txt
```

Figure 6: 3B Screenshot

Phase 4: Commits and History

Screenshot 4A: pes log Output

```
(base) kana@fedora:~/Downloads/daa-assignmen$ ./pes log
commit c52605e8911e5b3d0d712ff78a73cd5bfcbaacffae2be40b19a2ee97bc597816
Author: Shriniketh Kana <PES1UG24CS446>
Date:   1776413694

    Add farewell

commit bf3f7d6bf4ed73e7284042e5f7e25650c936e47742f2a1d66ebaac3a98a121f4
Author: Shriniketh Kana <PES1UG24CS446>
Date:   1776413694

    Update file.txt

commit c57cbfa38e038f4c9d957769ec6a6ca32e664105ca62665f60ae45f69fd21afb
Author: Shriniketh Kana <PES1UG24CS446>
Date:   1776413693

    Initial committial commit6A
```

Figure 7: 4A Screenshot

```
(base) kana@fedora:~/Downloads/daa-assignmen$ cat .pes/HEAD && cat .pes/refs/h
ref: refs/heads/main
c52605e8911e5b3d0d712ff78a73cd5bfcbaacffae2be40b19a2ee97bc597816
```

Figure 8: 4B Screenshot

```
(base) kana@fedora:~/Downloads/daa-assignmen$ find .pes/objects -type f | wc -
10
```

Figure 9: 4C Screenshot

Screenshot 4B: Object Growth

Screenshot 4C: References and HEAD

Final Phase: Full Integration Test

Analysis Questions (Phase 5 & 6)

Branching and Checkout

Q5.1: A branch in Git is just a file in `.git/refs/heads/` containing a commit hash. Creating a branch is creating a file. Given this, how would you implement `pes checkout <branch>` — what files need to change in `.pes/`, and what must happen to the working directory? What makes this operation complex?

Answer: To implement `pes checkout <branch>`, I would first verify that the specified branch file exists in `.pes/refs/heads/`. Then I would update `.pes/HEAD` to contain `ref: refs/heads/<branch>`. Lastly, I would read the target commit, load its tree snapshot, and overwrite the working directory files to match the snapshot. This operation is complex because we must safely replace files in the working directory without destroying untracked files or unstaged changes, and also properly transition the `.pes/index` file to match the new HEAD commit so the staging area is correctly synced with the newly checked-out branch.

Q5.2: When switching branches, the working directory must be updated to match the target branch's tree. If the user has uncommitted changes to a tracked file, and that file differs between branches, checkout must refuse. Describe how you would detect this "dirty working directory" conflict using only the index and the object store.

```
(base) kana@fedora:~/Downloads/daa-assignmen$ make test-integration
=== Running integration tests ===
bash test_sequence.sh
=== PES-VCS Integration Test ===

--- Repository Initialization ---
Initialized empty PES repository in .pes/
PASS: .pes/objects exists
PASS: .pes/refs/heads exists
PASS: .pes/HEAD exists

--- Staging Files ---
Status after add:
Staged changes:
  staged:     file.txt
  staged:     hello.txt

Unstaged changes:
  (nothing to show)

Untracked files:
  (nothing to show)


--- First Commit ---
Committed: 46218d61da55... Initial commit

Log after first commit:
commit 46218d61da55e2beef45ee51e474e99459ea0f2033f8a20d55f2788b61fde292
Author: Shriniketh Kana <PES1UG24CS446>
Date:   1776413695

    Initial commit


--- Second Commit ---
Committed: fc29a0fe243b... Update file.txt

--- Third Commit ---
Committed: 48ac3f5d6afa... Add farewell

--- Full History ---
commit 48ac3f5d6afaaee01053a0d66705c8527c67429526569d96b5574b9fe3cdab3b
Author: Shriniketh Kana <PES1UG24CS446>
Date:   1776413695

    Add farewell

commit fc29a0fe243b89c8166609fac296c5adbd261a5bd3a7c3ed2b6ee9737797b9f3
Author: Shriniketh Kana <PES1UG24CS446>
Date:   1776413695

    Update file.txt

commit 46218d61da55e2beef45ee51e474e99459ea0f2033f8a20d55f2788b61fde292
Author: Shriniketh Kana <PES1UG24CS446>
Date:   1776413695

    Initial committial commit6A


--- Reference Chain ---
HEAD:
ref: refs/heads/main
refs/heads/main:
48ac3f5d6afaaee01053a0d66705c8527c67429526569d96b5574b9fe3cdab3b

--- Object Store ---
Objects created:
10
.pes/objects/0b/d69098bd9b9cc5934a610ab65da429b525361147faa7b5b922919e9a23143
.pes/objects/10/1f28a12274233d119fd3c8d7c7216054ddb5605f3bae21c6fb6ee3c4c7cbf
.pes/objects/46/218d61da55e2beef45ee51e474e99459ea0f2033f8a20d55f2788b61fde29
.pes/objects/48/ac3f5d6afaaee01053a0d66705c8527c67429526569d96b5574b9fe3cdab3
.pes/objects/58/a67ed1c161a4e89a110968310fe31e39920ef68d4c7c7e0d6695797533f50
.pes/objects/ab/5824a9ec1ef505b5480b3e37cd50d9c80be55012cabe0ca572dbf95978829
```

Figure 10: Final Integration Test

Answer: We first iterate through the target branch's tree blobs. For each file, we compare its content hash against what is stored in the current `.pes/index`. If the entries disagree, we verify the physical file on disk using `stat()` checking if its metadata (size/mtime) differs from that of the index. If it differs, that indicates local uncommitted or unstaged changes. Alternatively, any file staged in the index with a blob hash that differs from both the current branch's commit tree and the target branch's checkout tree would instantly signal a conflict, forcing checkout to abort before overwriting.

Q5.3: "Detached HEAD" means HEAD contains a commit hash directly instead of a branch reference. What happens if you make commits in this state? How could a user recover those commits?

Answer: If you make commits in a "Detached HEAD" state, `.pes/HEAD` will be overwritten with the new commit hash, but no branch reference within `.pes/refs/heads/` will be updated to point to this new commit. If you later checkout a different branch, those commits become "unreachable" (dangling) since nothing tracks them. A user can recover them by inspecting terminal logs for the hashes or walking through `.pes/objects/` to manually extract the orphan commit's hash to establish a new branch reference (simulating standard git reflog recovery mechanisms).

Garbage Collection and Space Reclamation

Q6.1: Over time, the object store accumulates unreachable objects — blobs, trees, or commits that no branch points to (directly or transitively). Describe an algorithm to find and delete these objects. What data structure would you use to track "reachable" hashes efficiently? For a repository with 100,000 commits and 50 branches, estimate how many objects you'd need to visit.

Answer: I would build a Mark-and-Sweep garbage collection algorithm. First, I would use a `Hash Set` (or a probabilistic Bloom filter alongside a localized array) to track reachable hashes efficiently. In the "Mark" phase, I would iterate through every branch pointer in `.pes/refs/heads/` and traverse backward through their linked parent commits. For each commit, I add its hash to the set, then load its associated tree, recursively adding every child tree and linked blob hash to the set. In the "Sweep" phase, I would iterate through `.pes/objects/` and delete any file whose hash name isn't contained within the reachable set. For 100,000 commits, assuming roughly 10 uniquely modified trees/blobs per commit diff on average, we could visit around 1,000,000 distinct objects. Using a reliable hash set mapping for these object hashes ensures $O(1)$ lookups, optimizing the time taken substantially.

Q6.2: Why is it dangerous to run garbage collection concurrently with a commit operation? Describe a race condition where GC could delete an object that a concurrent commit is about to reference. How does Git's real GC avoid this?

Answer: A race condition occurs if `pes commit` relies on staging previously

written blobs up to a new commit tree object in the background. A running GC might scan the system's active refs and conclude those staged blobs are officially "unreachable" (as the commit hasn't completed to bind them historically to the HEAD reference yet). The GC then deletes the blobs maliciously just as the commit finishes writing referencing them, rendering a corrupted tree. Git natively avoids this via a two-week grace period restriction: It deliberately skips pruning unreachable objects created within the last 14 days. It also tightly coordinates index locks (e.g. `index.lock` configuration) mitigating simultaneous destructive commands.